Requirement 4

Design 1

Create class/interface:

1. CoatWeaponAction
2. Poisoning
3. Frosting
4. Snowcoating
5. Yewberrycoating
6. Coatable interface
7. Coating interface
8. CountinuousWarmthloss

Pro	Cons
Implementing the same interface, each coating only needs to define its own behaviour, like how it poisons or freezes the target.	If both YewberryCoating and SnowCoating implement the same Coating interface but still repeat similar internal logic, such as stacking, duration handling, and round decrementing, this design breaks the DRY principle and reduces code maintainability.
	Since ContinuousDamage already handles turn-based effects, creating another ContinuousWarmthLoss is unnecessary and breaks the DRY and Single Responsibility principles.
	Although new coatings can still implement the same Coating interface, the current design lacks a shared abstract layer for common logic like duration and stacking.

Design 2

Create class

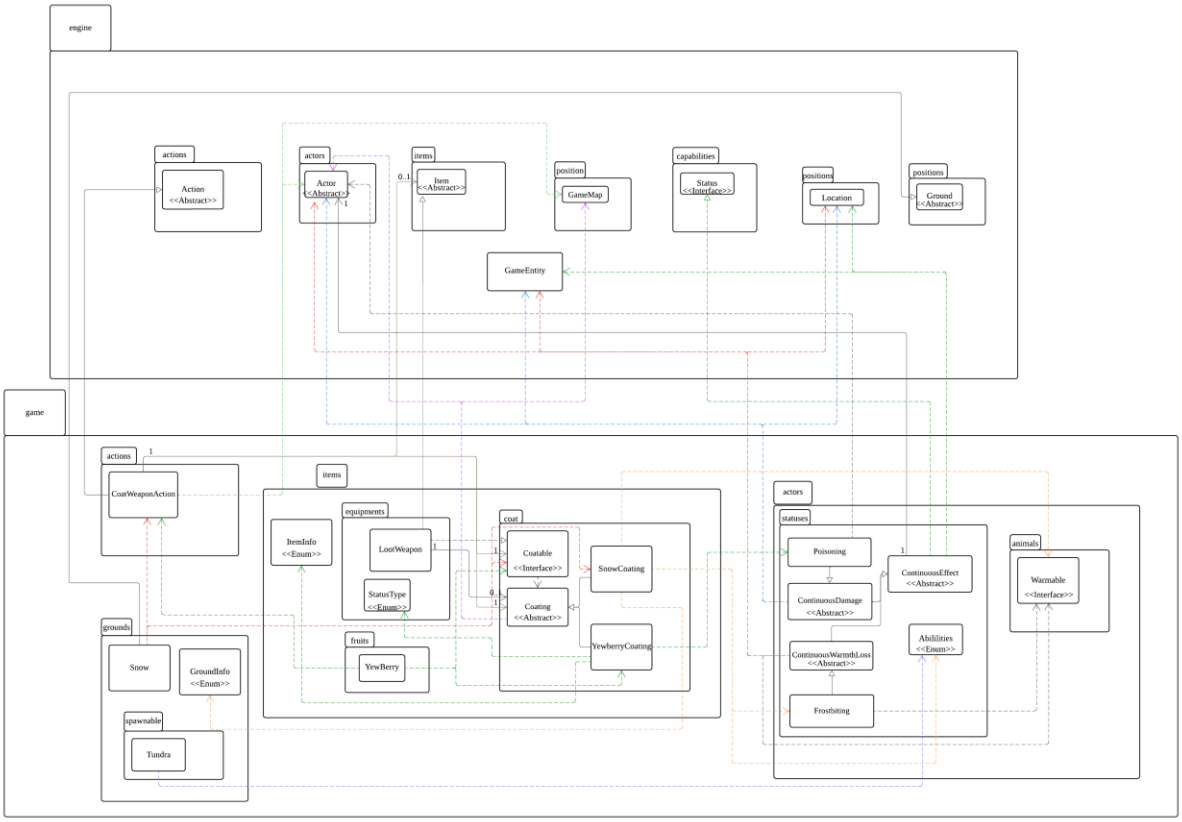
1. CoatWeaponAction
2. Poisoning
3. Frostbiting
4. CountinuousWarmthloss
5. CountinuousEffect
6. Snowcoating
7. YewBerrycoating
8. Coatable interface

9. Coating abstract

Pro	Cons
Using an abstract Coating class allows both YewberryCoating and SnowCoating to share common logic such as duration, stacking, and expiry, which followed the DRY and Open/Closed principles because new coatings can be added by extending the abstract class without changing existing code.	For the current scope with only two coatings, the design may seem slightly over-engineered. The abstract structure adds extra complexity at this stage, but its advantages—such as easier extension and better code reuse. it will become more valuable when more coating types or coatable targets are introduced in the future.
The design follows the DRY principle through a shared abstract class ContinuousEffect, which centralises common logic like ticking, duration handling, stacking, and expiry. Subclasses such as ContinuousDamage and ContinuousWarmthLoss only define how the effect is applied, promoting code reuse and consistent behaviour across continuous effects.	
The design maintains low coupling through the Dependency Inversion and Single Responsibility principles. CoatWeaponAction depends on abstract types Coating and Coatable instead of concrete classes. Weapons handle combat, coatings manage their effects, and the action coordinates them. Each class has a single, clear responsibility, making the system easier to maintain and extend.	

Choose:

Design 2 was chosen because it follows SOLID more rigorously, yielding a cleaner, more maintainable structure than Design 1. An abstract Coating and a Coatable interface separate concerns and remove duplication: coatings define effects, the action applies them, and weapons handle combat—SRP. New coatings are added by subclassing without touching existing code—OCP. DIP is met because CoatWeaponAction depends on abstractions (Coating, Coatable), not concrete types. It also respects LSP (any Coating subtype can replace another without breaking behavior) and ISP (the Coatable interface is minimal—only coating getters/setters—so weapons aren't forced to implement irrelevant methods). Although slightly more complex, this design delivers greater flexibility, scalability, and long-term maintainability, making it the right choice for REQ4.



1. What classes will exist in the extended system
    a. Interface:
        i. Coatable
        ii. Warmable
    b. Abstract class:
        i. ContinuousEffect
        ii. ContinuousWarmthloss
        iii. continuousDamage
        iv. Coating
    c. Enum class:
        i. Abilities
        ii. ItemInfo
        iii. GroundInfo
        iv. StatusType
    d. Concrete Class:
        i. CoatWeaponAction
        ii. SnowCoating
        iii. YewberryCoating
        iv. LootWeapon
        v. YewBerry

vi. Frosting
vii. Poisoning
viii. Snow
ix. Tundra

2. Roles & Responsibilities
    a. Coatable → Represents any object that can receive a coating, usually a weapon, and defines how it interacts with different coating effects in the game.
    b. ContinuousEffect → Abstract ticked-effect base centralising duration, stacking rules, and expiry; prevents duplication across different status implementations.
    c. ContinuousWarmthLoss → ContinuousEffect that decreases warmth each turn; models cold exposure/frostbite; stacks with other warmth drains.
    d. ContinuousDamage → similar with ContinuousWarmthLoss.
    e. Abilities → Stores unique capability flags for actors (e.g., COLD_RESISTANT).
    f. CoatWeaponAction → Menu action that applies a chosen coating to a Coatable; replaces any existing coating and optionally consumes the source item.
    g. SnowCoating → Extends the Coating class and applies a frostbite effect to targets, causing continuous warmth loss for a few turns after being attacked.
    h. YewberryCoating → Extends the Coating class and applies a poisoning effect to the target, dealing continuous damage for several turns after being hit.
    i. LootWeapon → Concrete weapon class supporting coatings; attack flow: base damage/effects, then delegate to current coating's applyOnHit.
    j. YewBerry → Acts as a consumable item that provides the YewBerryCoating when used, letting the player coat a weapon to apply poison effects on future attacks.
    k. Frostbiting → A continuous status effect that reduces an actor's warmth each turn, used by SnowCoating to simulate the cold damage caused by frostbite.
    l. Poisoning → A continuous status effect that deals damage over several turns, used by YewBerryCoating to represent the poison applied to targets after being hit.
    m. Snow → Represents a ground source that can be used to coat weapons with SnowCoating, allowing the player to apply frostbite effects on the weapons.
    n. Tundra →Spawning ground that grants spawned creatures COLD_RESISTANT (and extra HP), rendering them immune to Frostbiting from SnowCoating.

3. How these classes relate to and interact with the existing system.

    The coating system ties neatly into combat, items, and statuses while adhering to SOLID. Coatable works with LootWeapon so weapons accept coatings without changing their attack flow, and CoatWeaponAction (in the Action system) lets actors apply a coating from a consumable (YewBerry) or an environmental source (Snow). Via polymorphism, any Coatable can receive a coating.

An abstract Coating integrates with the continuous-effect framework, centralising duration, stacking, and expiry so it plugs directly into the engine's status tick cycle. Concrete coatings (YewBerryCoating, SnowCoating) delegate to statuses (Poisoning, Frostbiting) built on ContinuousDamage and ContinuousWarmthLoss (both inherit the same per-turn ticking from ContinuousEffect). Environment classes connect too: Tundra spawns COLD_RESISTANT creatures (immune to SnowCoating's frostbite), while Snow offers a natural coating source.
SOLID in practice: SRP (coatings handle effects, actions apply, weapons fight), OCP (add new coatings via subclassing), DIP (depend on Coating/Coatable abstractions), LSP (any Coating subtype is swappable), ISP (minimal Coatable interface: get/set coating only).

4. How the classes interact to deliver functionality.
   When the player coats a weapon, CoatWeaponAction runs: they choose YewBerry (consumed) or Snow (no item), the action verifies the weapon is Coatable (torches excluded), then replaces any existing coating and updates state. The abstract Coating centralises effect type, duration, and remaining turns (preserving weapon combat code—SRP, DIP). In combat, coatings trigger polymorphically: YewBerryCoating → Poisoning, SnowCoating → Frostbiting; these statuses use the tick framework (ContinuousDamage / ContinuousWarmthLoss ← ContinuousEffect) so per-turn effects stack and expire cleanly. Tundra-spawned creatures have COLD_RESISTANT, blocking frostbite; Snow is a renewable source, while YewBerry provides the consumable path. Overall flow—execute action → apply/replace via abstraction → engine ticks each turn—extends mechanics while staying OCP (add new coatings by subclassing), LSP (any Coating is swappable), and ISP (minimal Coatable), delivering a modular, maintainable design.